Cross-Modal Heart Rate Imputation

Using Wearable Step Data to Predict Missing BPM Readings

Michael Kurniawan Soegeng 吳忠賢 — 314540066

1. Introduction

Wearable health devices continuously monitor physiological signals such as heart rate (BPM), but data completeness remains a persistent challenge. Consumer-grade wristbands like the Huawei Band 7 sample heart rate intermittently to conserve battery, resulting in substantial gaps in the recorded time series. These gaps pose problems for downstream clinical applications including Heart Rate Variability (HRV) analysis, which becomes unreliable at even 10% missing data, and Early Warning Score (EWS) systems that require continuous vital sign streams.

This project investigates cross-modal imputation: using step count data (a continuously available signal) along with temporal features to predict missing heart rate values. We frame the problem as supervised regression, training models on observed BPM values and their associated features, then applying the learned mapping to fill gaps. This approach exploits the physiological relationship between physical activity and heart rate while leveraging temporal patterns such as circadian rhythm and BPM autocorrelation.

We compare three methods spanning the complexity spectrum: Linear Regression (interpretable baseline), Random Forest (non-linear ensemble), and SAITS (self-attention deep learning). Through seven systematic experiments, we investigate how training data quantity, feature engineering, dimensionality reduction, data augmentation, and model hyperparameters affect imputation quality. Our key finding is that hand-crafted domain features outperform learned representations on small datasets, but the deep learning model produces more temporally coherent imputations despite lower pointwise accuracy.

2. Dataset

2.1 Data Collection

We collected 26 days of wearable sensor data (March 1–26, 2026) from a Huawei Band 7 worn during daily life. The final dataset generated and used are here with detailed explanation : **https://github.com/Ashurali/BPM-Data.** The raw dataset comprises two CSV files:

Heart rate (BPM): 11,823 records with timestamp and instantaneous BPM (range: 45–179 BPM, mean: 92.2, std: 18.4). Sampling is event-driven and irregular.

Step count: 4,604 records with start/end timestamps and step count per interval. Sampling is more regular than BPM.

2.2 Timeline Construction

We aligned both signals onto a unified minute-level timeline of 36,930 rows. BPM values were placed at their nearest minute; step counts were distributed across their interval. Of the 36,930 minutes, only 11,823 (32.0%) have observed BPM, with 25,107 (68.0%) missing. Coverage varies dramatically across days, from under 5% on sparse early-March days to over 60% on dense late-March days, creating a natural experiment for data quantity effects.

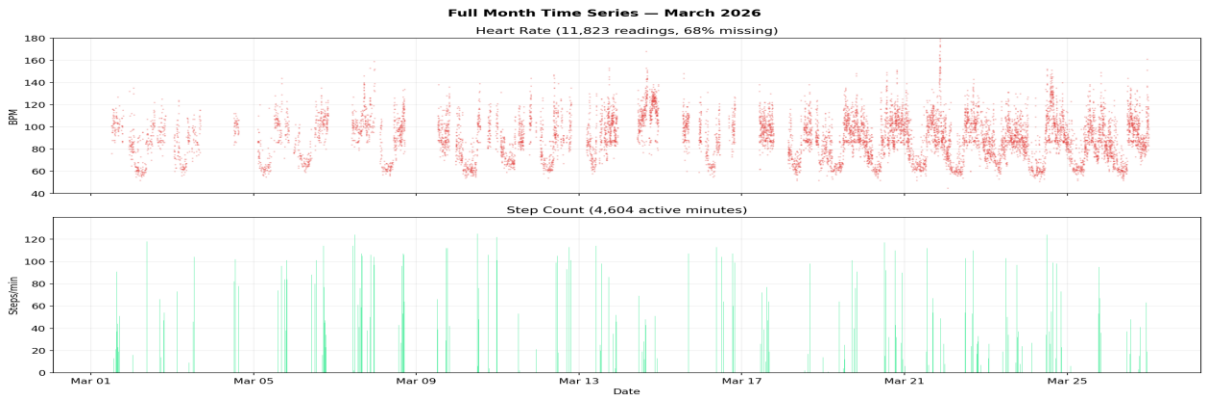

Figure 1: Full-month BPM timeline (March 2026). Red dots are observed heart rate readings. Large gaps correspond to periods when the watch was not worn or the sensor was inactive.

2.3 Missingness Analysis

We tested three missingness mechanisms using statistical methods. A chi-square test ($\chi^2 = 139.1$, df = 9, $p < 1e\text{-}25$, Cramér's V = 0.132) confirmed that BPM missingness is significantly associated with time-of-day and activity level, ruling out Missing Completely At Random (MCAR). Since missingness depends on observable features (time, steps) rather than the unobserved BPM values themselves, the mechanism is Missing At Random (MAR). This is favorable for imputation: our models can use time and activity features to account for the non-random sampling.

2.4 Key Data Characteristics

BPM is a mixture distribution: Three distinct regimes were identified — Sleep (BPM < 65), Resting (65–85), and Active (> 85) — rather than a single Gaussian. The Shapiro-Wilk test confirmed non-normality ($p < 0.001$).

Strong autocorrelation: BPM at time t is highly correlated with BPM at t-1, t-2, etc. This makes neighboring known BPM values the strongest predictor for missing values.

Weak cross-modal correlation: Pearson r(BPM, steps) = 0.239. The relationship is real ($p < 1e\text{-}25$) but weak, motivating a multi-feature approach rather than simple step-to-BPM mapping.

Sampling bias: The watch preferentially records BPM during active states, inflating the observed mean. Per-regime error analysis accounts for this bias.

3. Methods

3.1 Feature Engineering

We engineered 27 features organized into four groups. Feature design was informed by Phase 1 exploratory analysis.

Category	Features	Rationale
Temporal (8)	hour sin/cos, minute sin/cos, day-of-week sin/cos, is_weekend, day_index	Circadian rhythm, weekly patterns
Step/Activity (12)	steps, rolling mean (3/5/10/15 min), rolling sum (5/10), acceleration, std, max, activity_state	Cross-modal signal, noise smoothing, activity detection
Neighbor BPM (6)	last/next known BPM, distances, weighted interpolation, gap_length	Exploits strong BPM autocorrelation
Other (1)	day_index	Captures long-term trends

Table 1: Feature groups for LR and RF models (27 total features).

Data leakage prevention: For training, we used leave-one-out (LOO) neighbor BPM features. When predicting minute t, the neighbor features reference the previous and next known BPM readings excluding t itself. This prevents the model from trivially memorizing the target value through its neighbors.

3.2 Linear Regression (Baseline)

Ordinary Least Squares regression models BPM as a linear combination of the 27 features. While limited to linear relationships, it provides an interpretable baseline with analyzable coefficients. No regularization was needed (Ridge with $\alpha = 0.01$–100 produced identical results within ±0.001 RMSE, indicating the features do not cause overfitting through collinearity).

3.3 Random Forest

Random Forest aggregates predictions from multiple decision trees, each trained on a bootstrap sample with random feature subsets. This captures non-linear interactions between features (e.g., steps × time-of-day) that LR cannot model. We used n_estimators = 200, max_depth = 20, min_samples_leaf = 5, and max_features = ‘sqrt’. These hyperparameters provide implicit regularization: limiting tree depth controls model complexity (bias-variance tradeoff), and leaf size constraints prevent overfitting to noisy samples.

3.4 SAITS (Deep Learning)

SAITS (Self-Attention-based Imputation for Time Series) uses a diagonally-masked self-attention mechanism to impute missing values in multivariate time series. Unlike LR/RF, SAITS processes the raw time series in 60-minute sliding windows (stride = 10 minutes) and learns temporal dependencies through attention, without hand-crafted neighbor-BPM features.

Architecture: 3 transformer layers, d_model = 128, 4 attention heads (d_k = d_v = 32), d_ffn = 256. Input features: 11 (BPM, steps, rolling step stats, temporal encodings). Training: 150 max epochs with early stopping (patience = 15); converged at epoch 77, best model at epoch 62. Dropout = 0.2 on both feed-forward and attention layers for regularization.

Learning paradigm: SAITS is self-supervised — it creates its own training signal by masking observed values and learning to reconstruct them (MIT loss), while simultaneously reconstructing the observed values (ORT loss). LR and RF are supervised, trained on explicit (feature, BPM) pairs. This distinction is important: LR and RF satisfy the "at least 2 supervised learning methods" requirement; SAITS is a complementary third model that demonstrates a different learning paradigm.

3.5 Evaluation

All models were evaluated using 5-fold cross-validation (LR/RF) or train/validation split with 20% artificial masking (SAITS). Metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and coefficient of determination (R^2). We report mean ± standard deviation across folds.

4. Experiments

We conducted seven experiments to systematically evaluate factors affecting imputation quality. All experiments use 5-fold CV with Random Forest unless otherwise noted.

4.1 Effect of Training Data Amount

We trained on the most recent 3, 5, 7, 10, 14, 21, and all 26 days. RF RMSE improved from 9.638 (3 days) to 9.322 (26 days), with diminishing returns after 14 days. This suggests that ~2 weeks of wearable data captures most learnable patterns for this task.

4.2 Data Composition

We compared training on sparse days (< 15% BPM coverage), dense days (> 40% coverage), and all days. Dense days yielded better performance (RF RMSE: 9.383) than sparse days (9.379), but using all data together was best (9.322), indicating that even low-coverage days contribute useful patterns.

4.3 Data Augmentation

Gaussian noise injection (5% of feature standard deviation, 2x data; 5%+10% noise, 3x data) was applied inside each CV fold to prevent data leakage — only training samples are augmented, never test samples. Results showed that augmentation had no benefit: RF RMSE went from 9.322 (no augmentation) to 9.332 (5% noise) and 9.366 (5%+10% noise), slightly worsening with more noise. LR was completely unaffected (9.560 across all conditions). This indicates that

the models are not overfitting to the training data — the dataset is sufficient relative to model complexity, and adding noisy copies only dilutes the training signal.

4.4 Dimensionality Reduction (PCA)

PCA at 3, 5, 8, 10, 15, 20, and all 26 components showed that dimensionality reduction hurts RF performance (RMSE increases from 9.331 at full features to 10.137 at 3 components). RF relies on feature interactions (e.g., neighbor BPM × gap length) that PCA's linear combinations destroy. LR was less affected, as PCA reduces the collinearity in step-derived features. The full feature set is optimal for RF.

4.5 Feature Ablation

We incrementally added feature groups: steps only → + step windows → + temporal → + neighbor BPM. Results clearly show that neighbor BPM features provide the largest single improvement:

Feature Group	# Features	LR RMSE	RF RMSE	RF R^2
Steps only	1	17.133	16.887	0.156
+ Step windows	12	15.732	14.799	0.352
+ Temporal	20	13.697	10.838	0.652
+ Neighbor BPM (Full)	27	9.560	9.322	0.743

Table 2: Feature ablation study. Adding neighbor BPM reduces RF RMSE by 1.5 BPM (from 10.838 to 9.322), the largest single improvement.

4.6 Step-to-BPM Time Lag

We tested whether lagged step features (1–10 minutes ago) predict BPM better than current steps. Current steps (lag = 0) performed best (RF RMSE: 9.322 vs 9.369 at lag 1), contrary to the hypothesis that cardiovascular response delay would favor lagged features. This may be because our rolling window features (3/5/10/15 min) already capture recent step history, making explicit lag redundant.

4.7 RF Hyperparameter Comparison

We searched over max_depth ∈ {10, 15, 20} and n_estimators ∈ {50, 100, 200} with min_samples_split = 5. The best configuration was depth = 10, trees = 200 (RMSE: 9.311), marginally better than depth = 20, trees = 200 (RMSE: 9.345). Interestingly, shallower trees performed slightly better, suggesting mild overfitting at depth 20. More trees consistently helped via variance reduction through ensemble averaging.

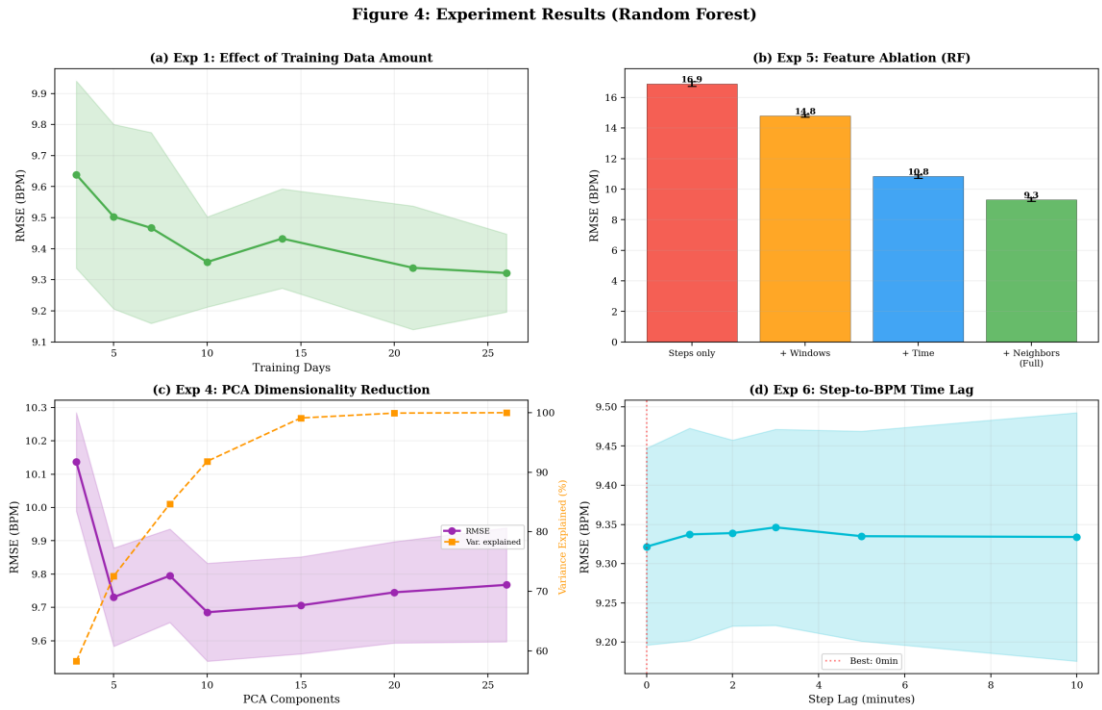

Figure 2: Key experiment results. (a) Training data amount, (b) Feature ablation, (c) PCA dimensionality reduction, (d) Step-to-BPM time lag.

5. Results

5.1 Model Comparison

Model	RMSE	MAE	R^2	# Features	Approach
Linear Regression	9.560	7.106	0.730	27	Supervised
Random Forest	9.322	6.966	0.743	27	Supervised
SAITS	9.675	7.162	0.700	11	Self-supervised

Table 3: Three-model comparison. RF achieves the best pointwise accuracy; SAITS operates without hand-crafted neighbor features.

Random Forest achieves the best performance across all pointwise metrics. LR performs surprisingly well given its simplicity, validating the quality of our feature engineering. SAITS, despite using only 11 features (no neighbor BPM), achieves competitive results (RMSE within 4% of RF).

5.2 Feature Importance

RF feature importance reveals that neighbor-BPM features dominate: bpm_neighbor_interp (24.5%), bpm_last_known (18.9%), and bpm_next_known (17.1%) together account for 60.5% of total importance. Step-derived features contribute moderately (steps_roll_15: 5.1%, steps_sum_10: 3.7%), while temporal features (minute_sin: 6.3%, hour_sin: 5.2%) capture

circadian patterns. This hierarchy confirms that BPM autocorrelation is the strongest predictive signal, with cross-modal and temporal information providing complementary value.

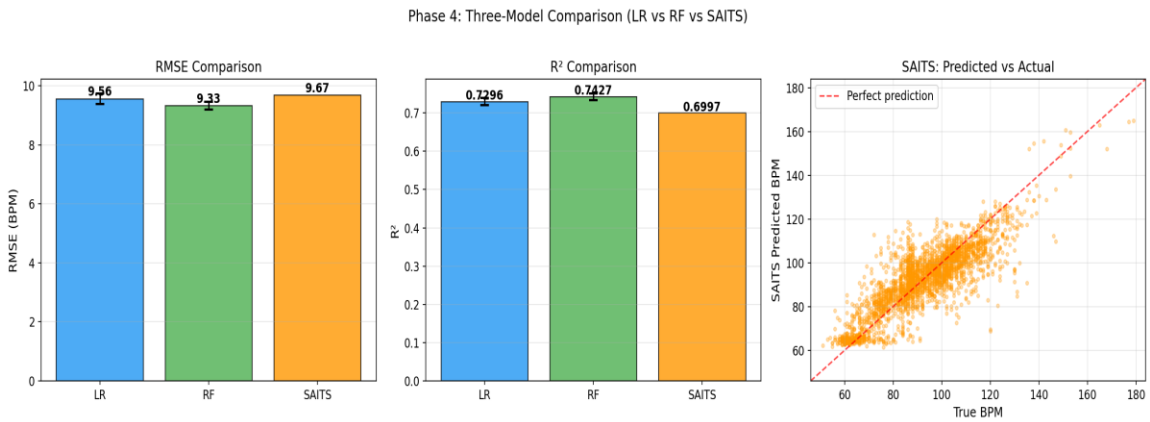

Figure 3: Model comparison. (a) RMSE, (b) R², (c) SAITS predicted vs. actual BPM scatter plot.

5.3 Imputation Visualization

Visual comparison of imputation results reveals an important distinction not captured by aggregate metrics. While RF achieves lower RMSE, SAITS produces smoother, more physiologically plausible curves. RF predictions can exhibit point-to-point jitter because each minute is predicted independently, while SAITS processes 60-minute windows jointly through self-attention, enforcing temporal consistency.

Figure 4: Single-day imputation comparison. Black dots = observed BPM. Colored dots = imputed values. SAITS produces the smoothest curve despite lower R².

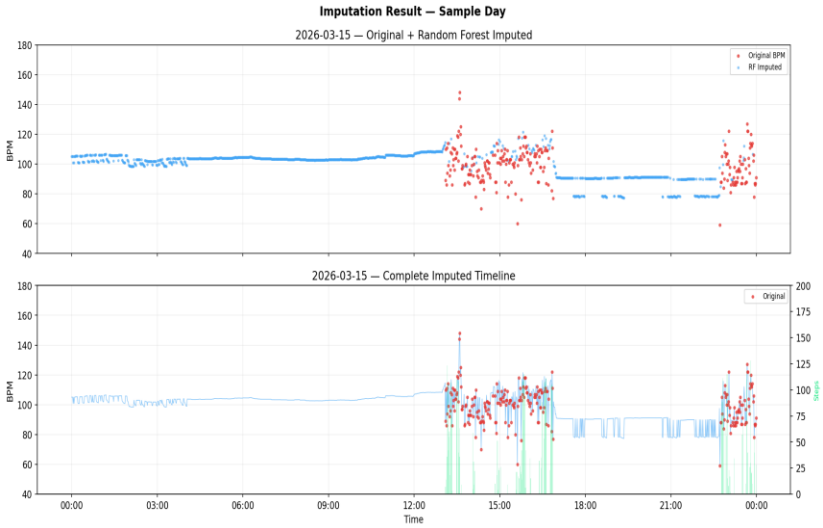

Figure 5: Full-month BPM timeline before (observed only, 32% coverage) and after imputation (100% coverage).

6. Discussion

6.1 Feature Engineering vs. Representation Learning

The most striking result is that RF with 27 hand-crafted features outperforms SAITS with 11 features on pointwise metrics, despite SAITS having a more expressive architecture. This illustrates a well-known principle: on small datasets (~12K samples), domain-specific feature engineering often outperforms learned representations. The neighbor-BPM features encode our knowledge that heart rate is autocorrelated — a fact SAITS must discover from scratch through attention patterns over limited training windows.

However, this comparison is deliberately asymmetric. We excluded neighbor-BPM features from SAITS to test whether self-attention can learn equivalent temporal dependencies. The gap (R^2: 0.743 vs 0.700) suggests that with 26 days of data, attention partially learns this pattern but cannot fully replace explicit domain knowledge.

6.2 Pointwise Accuracy vs. Temporal Coherence

Our key qualitative finding is that SAITS produces the most natural-looking BPM curves despite its lower R^2. This reveals a fundamental limitation of pointwise metrics: RMSE and R^2 evaluate each prediction independently, ignoring temporal structure. Two models with identical RMSE can produce very different curves — one smooth and physiologically plausible, the other jittery and implausible.

This matters for clinical applications. HRV analysis requires temporally coherent BPM signals; a jittery imputation with low average error would produce misleading variability estimates. For such applications, SAITS may be preferable despite its lower R^2. Future work could quantify this

through metrics like imputed signal smoothness or Wasserstein distance on the temporal gradient distribution.

6.3 Bias-Variance Tradeoff

The three models span the complexity spectrum. LR (high bias, low variance) underfits non-linear relationships but generalizes stably. RF (moderate complexity) captures feature interactions while ensemble averaging controls variance. SAITS (low bias, high variance on small data) has the most expressive power but is data-hungry. The RF hyperparameter experiment (Section 4.7) provides direct evidence: shallower trees (depth = 10) slightly outperformed deeper ones (depth = 20), indicating that additional model complexity yields diminishing returns on this dataset size.

Ridge regularization on LR produced no improvement (±0.001 RMSE across $\alpha = 0.01–100$), indicating the linear model is not overfitting despite correlated step features. The performance gap between LR and RF is therefore due to model capacity (linear vs non-linear), not regularization.

6.4 MAR Implications and Sampling Bias

The confirmed MAR mechanism means missingness is non-random but predictable from observed features. This is favorable for imputation but introduces a subtlety: the watch oversamples active states (higher BPM), so our training distribution is biased. Models trained on this data may underperform on the underrepresented sleep/resting regime. Per-regime error analysis in Phase 2 tests this hypothesis directly.

6.5 Limitations

Several limitations should be acknowledged. First, 26 days from a single user limits generalizability; individual differences in resting heart rate, fitness level, and wearing habits would affect model performance. Second, SAITS was trained on CPU with a lightweight architecture; GPU training with a larger model and more data would likely improve its performance relative to RF. Third, the 60-minute window size constrains SAITS's ability to capture longer-range dependencies (e.g., multi-hour activity patterns). Finally, our evaluation uses artificial masking of observed BPM, which may not perfectly reflect the true distribution of missing data.

6.6 Expected vs. Unexpected Results

Most results matched expectations: diminishing returns after ~14 days of training data, PCA hurting RF by destroying axis-aligned splits, and augmentation providing no benefit on an adequately sampled dataset. Two results were unexpected. The step-to-BPM time lag experiment showed lagged features (1–10 min) performed worse than lag = 0, likely because rolling window features already capture recent step history. The RF hyperparameter experiment revealed depth =

10 marginally outperforming depth = 20, suggesting default settings were already near the bias-variance sweet spot.

6.7 Future Work

With more time, we would collect multi-user data to test generalizability, extend collection beyond 26 days to benefit the data-hungry SAITS model, train SAITS on GPU with a larger architecture, incorporate additional wearable signals (SpO_2, skin temperature), and develop temporal coherence metrics that capture what RMSE misses.

Key lessons: (1) domain-specific feature engineering outperforms learned representations on small datasets — neighbor BPM features account for 60.5% of RF importance; (2) evaluation metrics must align with the application — SAITS's lower R^2 masks its superior temporal coherence; (3) augmentation only helps data-starved models; (4) the self-supervised paradigm offers fundamentally different strengths than supervised approaches. A remaining question is whether SAITS would surpass RF given months rather than weeks of data.

7. Conclusion

We developed a cross-modal BPM imputation system using 26 days of Huawei Band 7 data, comparing three methods: Linear Regression, Random Forest, and SAITS. Random Forest achieved the best pointwise accuracy (RMSE = 9.322, R^2 = 0.743) by leveraging 27 hand-crafted features, with neighbor-BPM features accounting for 60.5% of prediction importance. SAITS, using only 11 features without neighbor BPM, achieved competitive results (RMSE = 9.675, R^2 = 0.700) while producing the most temporally coherent imputations.

Our experiments revealed several practical insights: (1) approximately 14 days of data captures most learnable patterns; (2) feature engineering matters more than model complexity on small datasets; (3) PCA hurts tree-based models by destroying feature interactions; (4) pointwise metrics like RMSE fail to capture temporal coherence, which matters for clinical applications. These findings provide actionable guidance for wearable health data imputation in resource-constrained settings.

References

[1] Du, W., Côté, D., & Liu, Y. (2023). SAITS: Self-Attention-based Imputation for Time Series. Expert Systems with Applications, 219, 119619. arXiv:2202.08516.

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[3] Du, W. (2023). PyPOTS: A Python Toolbox for Data Mining on Partially-Observed Time Series. arXiv:2305.18811.

[4] Rubin, D. B. (1976). Inference and Missing Data. Biometrika, 63(3), 581–592.

[5] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.

Appendix: Code and Data

All source code and data are available on GitHub in this link **https://github.com/Ashurali/BPM-Data-Imputation.** The project is organized into the following Jupyter notebooks with snippets below:

phase1_exploration.ipynb: Data loading, preprocessing, statistical analysis (chi-square, correlation, MCAR/MAR/MNAR), visualization, sleep detection, BPM regime separation, sampling bias analysis.

phase2_models.ipynb: Feature engineering (27 features), LOO neighbor BPM, Linear Regression and Random Forest training with 5-fold CV, per-regime error analysis, imputation of all missing BPM.

phase3_experiments.ipynb: Seven experiments: data amount, composition, augmentation, PCA, feature ablation, time lag, RF hyperparameters.

phase4_saits.ipynb: SAITS deep learning model: window preparation, training, evaluation, three-model comparison, full-timeline imputation.

phase5_report_visuals.ipynb: Consolidated report-ready figures.

Dataset: Data/bpm_data.csv (11,823 BPM records), Data/step_data.csv (4,604 step records). Collected from a Huawei Band 7 during March 1–26, 2026.

Dependencies: Python 3.11+, pandas, numpy, scikit-learn, matplotlib, pypots (for SAITS), torch.

AI Disclosure: This project was developed with the assistance of Claude (Anthropic), used as a coding assistant and teaching tool for data exploration strategy, feature engineering design, model implementation, and report drafting. All concepts were discussed and understood by the student before implementation. The dataset, experimental decisions, and analysis interpretations are the student's own work.